

INHERITANCE

- Inheritance is one of the most important features of **Object-Oriented Programming (OOP)**.
- Inheritance is the process by which one class acquires the properties and behaviours (variables and methods) of another class.
 - The existing class is called **Superclass / Parent class / Base class**
 - The new class is called **Subclass / Child class / Derived class**
- Inheritance represents the **IS-A relationship** which is also known as a *parent-child* relationship.
- **Purpose:**
 - **Code Reusability:** Inheritance allows a subclass to reuse the properties and methods of a superclass, reducing code duplication.
 - **Improves maintainability:** Inheritance improves maintainability by allowing changes to be made in the superclass, which are automatically reflected in all subclasses.
 - **Supports method overriding (runtime polymorphism):** Inheritance allows a subclass to provide a specific implementation of a superclass method, enabling runtime polymorphism.
 - **Helps in hierarchical classification:** Inheritance allows classes to be organized in a hierarchical structure based on their relationships, improving clarity and logical representation.

General Syntax:

- Java uses the keyword “**extends**” to implement inheritance

General Syntax:

```
class Parent {  
    // properties and methods  
}  
  
class Child extends Parent {  
    // additional properties and methods  
}
```

Example of Inheritance:

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats food");  
    }  
}  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
public class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat(); // inherited method  
        d.bark(); // own method  
    }  
}
```

Output:

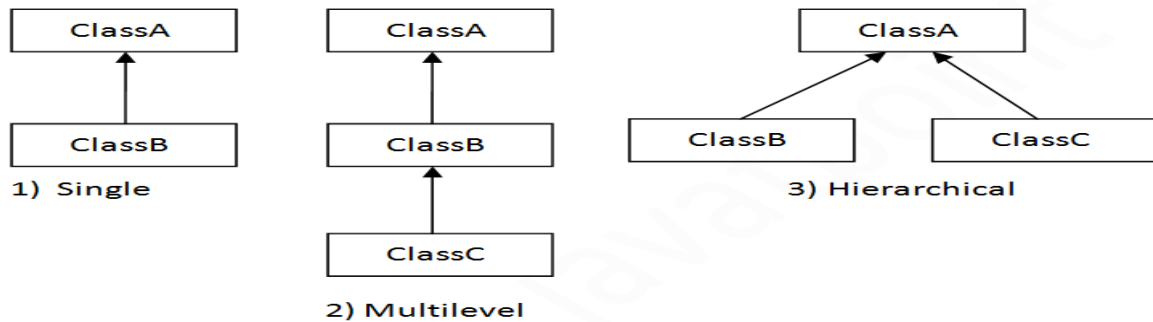
Animal eats food
Dog barks

- Private members of a superclass are inherited but cannot be accessed directly in the subclass; they can be accessed only through public or protected methods of the superclass.

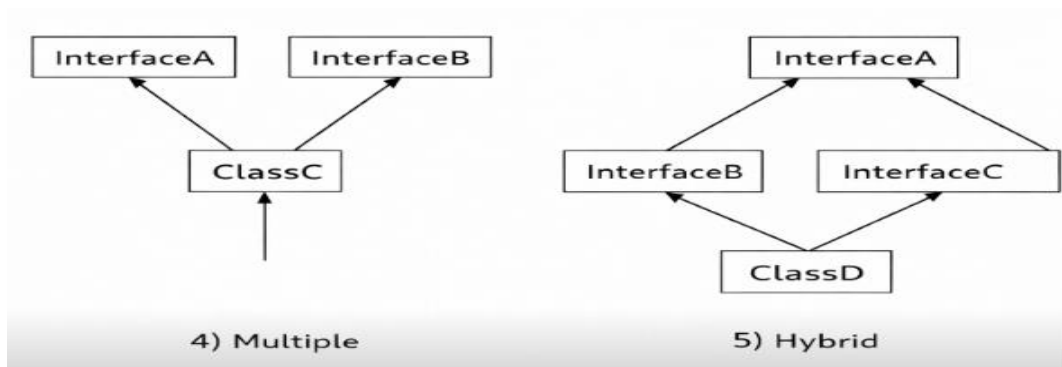
```
class Animal {  
    private String name = "Dog"; // private variable  
    // public setter method to set name  
    public void setName(String name) {  
        this.name = name;  
    }  
    // public method to access private variable  
    public String getName() {  
        return name;  
    }  
}  
class Dog extends Animal {  
    void display() {  
        // System.out.println(name); // ERROR: cannot access private member directly  
        // Access through public method  
        System.out.println("Animal name is: " + getName());  
    }  
}  
public class InheritanceDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.display();  
    }  
}
```

Types of Inheritance

- Java supports 5 types of Inheritance conceptually, but only some directly with classes.
- On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.



- In Java, Multiple and hybrid inheritance is supported through interface concept only



Single Inheritance:

- Single inheritance occurs when one subclass inherits from only one superclass.

Example:

```
class A {  
    void methodA() {  
        System.out.println("in Class A methodA() method");  
    }  
}  
class B extends A {  
    void methodB() {  
        System.out.println("in Class B methodB() method ");  
    }  
}
```

Multilevel Inheritance:

- Multilevel inheritance occurs when a class inherits from another class, which itself inherits from another class.

Example:

```

class A {
    void methodA() {
        System.out.println("in Class A methodA() method ");
    }
}
class B extends A {
    void methodB() {
        System.out.println("in Class B methodB() method ");
    }
}
class C extends B {
    void methodC() {
        System.out.println("in Class C methodC() method ");
    }
}

```

Hierarchical Inheritance:

- Hierarchical inheritance occurs when multiple subclasses inherit from the same superclass.

Example:

```

class A {
    void methodA() {
        System.out.println("in Class A methodA() method ");
    }
}
class B extends A {
    void methodB() {
        System.out.println("in Class B methodB() method ");
    }
}
class C extends A {
    void methodC() {
        System.out.println("in Class C methodC() method ");
    }
}

```

Multiple Inheritance (Not supported with classes):

- Multiple inheritance occurs when one class inherits from more than one superclass.
- Java does not support multiple inheritance with classes to avoid ambiguity (Diamond Problem). It is supported using interfaces.

Not allowed:

```

class A {}

class B {}

class C extends A, B {} // ERROR

```

Supported using interfaces.

Example:

```
interface A {  
    public abstract void methodA() ;  
}  
  
interface B {  
    public abstract void methodB() ;  
}  
  
class C implements A, B{  
    public void methodA() {  
        System.out.println("in Class C methodA() method ");  
    }  
  
    public void methodB() {  
        System.out.println("in Class C methodB() method ");  
    }  
}
```

Hybrid Inheritance:

- Combination of multiple and hierarchical inheritance.
- Java supports hybrid inheritance only through interfaces.

Example:

```
interface A {  
    public abstract void methodA() ;  
}  
  
interface B extends A {  
    public abstract void methodB() ;  
}  
  
interface C extends A {  
    public abstract void methodC() ;  
}  
  
class D implements B, C{  
    public void methodA() {  
        System.out.println("in Class D methodA() method ");  
    }  
    public void methodB() {  
        System.out.println("in Class D methodB() method ");  
    }  
    public void methodC() {  
        System.out.println("in Class D methodC() method");  
    }  
}
```

super Keyword

- The super keyword in Java is a reference variable that is used to refer to the immediate parent class object.
- It allows a subclass to access the variables, methods, and constructors of its parent class.
- The super keyword is mainly used when the subclass and superclass have members with the same name, and the subclass needs to explicitly refer to the parent class version.

Access Parent Class Variable:

- The super keyword is used to access the variable of the parent class when both parent and child classes have variables with the same name.
- It helps avoid confusion between superclass and subclass variables.

```
class A {  
    int x = 10;  
}  
class B extends A {  
    int x = 20;  
    void display() {  
        System.out.println(super.x); // parent variable  
        System.out.println(x);      // child variable  
    }  
}  
  
    B b1=new B();  
    b1.display();
```

Output:

```
10  
20
```

Access Parent Class Method:

- The super keyword is used to call a method of the parent class when the subclass has overridden the same method.
- It allows the subclass to access the original implementation of the parent class method.

```
class A {  
    void display() {  
        System.out.println("Parent method");  
    }  
}  
class B extends A {  
    void display() {  
        System.out.println("Child method");  
    }  
}
```

```

    }
    void show() {
        super.display();           // calls parent class method
        display();                 // calls child class method
    }
}

    B b1=new B();
    b1.show();

```

Output:

Parent method
Child method

Access Parent class Constructor:

- The super keyword is used to call the constructor of the parent class from the subclass constructor.
- It allows the subclass to initialize the properties and members inherited from the parent class.
- The super keyword ensures that the parent class constructor is executed before the subclass constructor. This helps in properly initializing the parent class data members.
- The call to the parent class constructor using **super()** **must always be the first statement in the subclass constructor**.
- If the first statement in the subclass constructor is neither a call to **super()** nor **this()**, the Java compiler automatically inserts a call to the parent class default constructor **super()** as the first statement.

Example:

```

class A {
    int num1;

    A(int i) {
        num1 = i;
        System.out.println("Parent constructor");
    }
}

class B extends A {
    int num2;
    B(int i, int j) {
        super(i);
        num2 = j;
        System.out.println("Child constructor");
    }
    void displayValues() {
        System.out.println("num1 = " + num1);
        System.out.println("num2 = " + num2);
    }
}

```

```
}  
}
```

```
B b1=new B(10,20);  
b1.displayValues();
```

Output:

```
Parent constructor  
Child constructor  
num1=10  
num2=20
```

Example:

```
class A {  
    A() {  
        System.out.println("A default constructor");  
    }  
}  
class B extends A {  
    int num1;  
    B() {  
        num1=10;  
        System.out.println("B default constructor");  
    }  
    B(int i) {  
        this();  
        num1=i;  
        System.out.println("B one arg constructor");  
    }  
}  
B b1 = new B(20);
```

Output:

```
A default constructor  
B default constructor  
B one arg constructor
```

Method Overriding

- Method overriding is a feature of inheritance in Java where a subclass provides its own implementation of a method that is already defined in its superclass.
- The overridden method in the subclass has the same name, same return type, and same parameters as the method in the parent class.
- Method overriding is used to achieve runtime polymorphism, which means the method that gets executed is determined at runtime based on the object type.

- In method overriding, the parent class defines a general method, and the child class modifies or extends that method to provide specific behaviour. This allows different classes to respond differently to the same method call.
- Method overriding occurs only when there is an inheritance relationship between two classes. Without inheritance, method overriding is not possible.
- When a method is overridden, the subclass method replaces the superclass method for the subclass object. However, the superclass method can still be accessed using the super keyword.
- Method overriding improves flexibility and allows programmers to reuse existing code while providing specialized behaviour in subclasses.

Method Overriding rules:

- The method signature and return type must be the same in both superclass and subclass.
- The subclass method can increase access level but cannot decrease it.
- The subclass method can throw the same or subclass exceptions of what a super class method throws, but cannot throw broader checked exceptions.

Example:

```
class Animal {
    public void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}

public class MethodOverridingDemo {
    public static void main(String[] args) {
        Animal a1 = new Animal();    // create Animal object
        a1.sound();                  // calls Animal class method
        Dog d = new Dog();           // create Dog object
        d.sound();                   // calls Dog class method (overridden method)
    }
}
```

Output:

```
Animal makes sound
Dog barks
```

In this example, the Dog class overrides the sound() method of the Animal class.

Dynamic Method Dispatch

- Dynamic Method Dispatch is the mechanism by which a superclass reference variable calls the overridden method of a subclass object at runtime, enabling runtime polymorphism in Java.
- Dynamic Method Dispatch occurs when a superclass reference variable refers to a subclass object, and the overridden method in the subclass is called using the superclass reference.
- In this mechanism, the type of the object determines which method is executed, not the type of the reference variable. This means the method call is decided at runtime based on the actual object, not the reference type.
- Dynamic Method Dispatch allows Java to support runtime polymorphism, which enables one interface to be used for different implementations.
- Dynamic Method Dispatch works only for overridden methods and not for overloaded methods. Method overriding must exist between superclass and subclass for dynamic method dispatch to occur.
- Dynamic Method Dispatch improves flexibility and extensibility in Java programs because new subclasses can be added without changing existing code.
- Dynamic Method Dispatch is also known as runtime method binding or late binding.

Example:

```
// Superclass
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}

// Subclass 1
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

// Subclass 2
class Rectangle extends Shape {
    void draw() {
```

```

        System.out.println("Drawing a Rectangle");
    }
}
// Subclass 3
class Triangle extends Shape {
    void draw() {
        System.out.println("Drawing a Triangle");
    }
}
// Main class
public class DynamicDispatchShapes {
    public static void main(String[] args) {
        Shape s; // superclass reference
        s = new Circle(); // refers to Circle object
        s.draw(); // calls Circle's draw()
        s = new Rectangle(); // refers to Rectangle object
        s.draw(); // calls Rectangle's draw()
        s = new Triangle(); // refers to Triangle object
        s.draw(); // calls Triangle's draw()
    }
}

```

- A superclass reference variable can point to a subclass object, but it can access only the methods and variables declared in the superclass, not the specific members of the subclass.

```

class Shape {
    void draw() {
        System.out.println("Drawing shape");
    }
}
class Circle extends Shape {
    void color() {
        System.out.println("Color is red");
    }
}
public class Test {
    public static void main(String[] args) {
        Shape s = new Circle(); // superclass reference, subclass object
    }
}

```

```

        s.draw(); // allowed (common method)

        // s.color(); // ERROR: cannot access subclass-specific method
    }
}

```

Using final with inheritance

- **Final Method:**

- A final method is a method that **cannot be overridden** by subclasses.
- **Characteristics:**
 - Can be inherited.
 - Cannot be overridden.
 - Used to prevent modification of method logic.

- **Example:**

```

class Parent {
    final void display() {
        System.out.println("Parent display");
    }
}

class Child extends Parent {
    // void display() { } ❌ Not allowed
}

```

- **Final class:**

- A final class in Java is a class that cannot be inherited by any other class. When a class is declared as final, it prevents other classes from extending it.
- **Example:**

```

final class Utility {
    void show() {
        System.out.println("Utility class");
    }
}

// class Test extends Utility ❌ Not allowed
{
    -----
}

```

ABSTRACT CLASSES

Abstract Method:

- An abstract method is a method that is declared using the abstract keyword and does not have a method body.

Syntax:

```
abstract returnType methodName();
```

Example:

```
abstract void sound();
```

- If a class contains at least one abstract method, then the class must be declared as abstract.

Example:

```
abstract class Animal {  
    abstract void sound(); // abstract method  
}
```

- The responsibility of implementing the abstract method lies with the subclass.

Abstract Class:

- An abstract class in Java is a class that is declared using the **abstract keyword**, is used as a base class for other classes, and can contain both **abstract methods (methods without a body)** and **concrete methods (methods with implementation)**, where the abstract methods must be implemented by its subclasses.
- Syntax of Abstract Class

```
abstract class ClassName {  
    // abstract methods  
    // concrete methods  
}
```

Example:

```
abstract class Animal {  
    abstract void sound();  
    void eat(){  
        System.out.println("Animal eats food");  
    }  
}
```

- An abstract class cannot be instantiated, which means objects cannot be created directly from an abstract class.
- An abstract class is mainly used to provide **a common structure and common behaviour for its subclasses**. It allows subclasses to share common code while also forcing them to implement specific methods.
- Abstract classes are used to achieve **abstraction**, which means hiding implementation details and showing only essential features to the user.
- **Subclass of Abstract Class:**
 - A subclass that extends an abstract class must provide implementation for all abstract methods. If the subclass does not implement all abstract methods, then it must also be declared as abstract.

Example:

```

abstract class Animal {
    abstract void sound();        // Abstract method 1
    abstract void move();        // Abstract method 2
    void eat() {
        System.out.println("Animal eats food");
    }
}

abstract class Cat extends Animal {
    // Implementing only one abstract method
    void sound() {
        System.out.println("Cat meows");
    }

    // move() is not implemented, so this class must be abstract
}

// Concrete subclass implementing both methods
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
    void move() {

```

```

        System.out.println("Dog runs");
    }
}
// Main class
public class AbstractDemo {
    public static void main(String[] args) {
        // Creating object of Dog class (allowed)
        Dog d = new Dog();
        d.sound();
        d.move();
        d.eat();
        // Creating object of Cat class (NOT allowed)
        // Cat c = new Cat(); // Compile-time error
        System.out.println("Cannot create object of abstract class Cat");
    }
}

```

- An abstract class can contain constructors, variables, static methods, and final methods.
- The abstract class is used to enforce method implementation in subclasses.
- Abstract classes cannot be declared as final.
- An Abstract class can be used as reference.

Example:

```

abstract class Animal {
    abstract void sound();
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
public class Test {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}

```

This supports runtime polymorphism.

- **Difference Between Abstract Class and Concrete Class**

Abstract Class	Concrete Class
Cannot create object	Can create object
May contain abstract methods	No abstract methods
Must be extended	Can be used directly
Uses abstract keyword	No abstract keyword

- **Advantages of Abstract Class**

- Achieves abstraction
- Supports code reuse
- Improves maintainability
- Supports polymorphism
- Provides common template

INTERFACES

- An interface in Java is a blueprint of a class that **contains abstract methods and constants**.
- It is used to achieve **complete abstraction** and also supports **multiple inheritance** in Java.
- An interface specifies **what a class must do**, but it does not describe **how the class will do it**. This allows developers to hide implementation details and focus only on required behavior, making programs easier to understand and manage.
- Interfaces help in creating **loosely coupled systems**. This means different parts of a program depend on abstractions rather than concrete classes. As a result, changes in one class do not strongly affect other classes, which improves flexibility and maintainability.
- In Java, the keyword used to create an interface is **“interface”**.

Syntax of Interface:

```
interface InterfaceName {
    // constants
    // abstract methods
}
```

Example:

```
interface Animal {  
    void sound();  
}
```

- **Implementing an Interface**

- A class implements an interface using the “**implements**” keyword. When a class implements an interface, it must provide implementations for all the abstract methods declared in that interface and we can create objects for those classes.

```
interface Animal {  
    void sound();  
    void eat();  
}  
  
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
  
    public void eat() {  
        System.out.println("Dog eats food");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog(); // Object creation allowed  
        d.sound();  
        d.eat();  
    }  
}
```

Here, the Dog class implements all methods (sound() and eat()) declared in the Animal interface. Therefore, Dog becomes a concrete class, and objects can be created.

- When a class does not implement all the abstract methods of the interface, then that class must be declared as an **abstract class**, and objects cannot be created for that class directly. Only concrete subclasses that implement all remaining methods can be instantiated.

```
interface Animal {  
    void sound();  
    void eat();  
}
```

```

abstract class Cat implements Animal {
    public void sound() {
        System.out.println("Cat meows");
    }
    // eat() method not implemented
}

```

The Cat class implements only one method (sound()) and does not implement eat(). Therefore, it must be declared as an **abstract class**. Since it is abstract, we cannot create objects directly:

```

Cat c = new Cat(); // ERROR – Cannot instantiate abstract class

```

```

class BabyCat extends Cat {
    public void eat() {
        System.out.println("Cat drinks milk");
    }
}

```

```

public class Demo {
    public static void main(String[] args) {
        BabyCat b = new BabyCat(); // Object creation allowed
        b.sound();
        b.eat();
    }
}

```

The BabyCat class implements the remaining method (eat()), so it becomes a concrete class, and objects can be created.

Important Rules of Interface

Rule 1: Interface cannot be instantiated

- An interface cannot be used to create objects directly.

Wrong:

```

Animal a = new Animal(); // ERROR

```

Correct:

```

Dog d = new Dog();

```

Rule 2: All methods are public and abstract by default

- All methods inside an interface are automatically considered public and abstract even if these keywords are not written explicitly. Therefore, writing:

Example:

```

interface Test {

```

```
        void show();
    }
    is same as:
    interface Test {
        public abstract void show();
    }
```

Rule 3: Variables are public, static, and final by default

- All variables declared inside an interface are constants by default.

For example:

```
    interface Test {
        int x = 10;
    }
```

is internally treated as:

```
    public static final int x = 10;
```

Because they are final, their values cannot be changed.

Rule 4: Class must implement all interface methods

When a class implements an interface, it is compulsory to implement all the methods declared inside that interface.

Example:

```
interface A {
    void show();
}
class B implements A {
    public void show() {
        System.out.println("Implemented");
    }
}
```

- **Multiple Inheritance using Interface**

Java supports multiple inheritance through interfaces. A class can implement more than one interface at the same time.

Example:

```
interface A {
    void methodA();
}
interface B {
    void methodB();
}
```

```

    }
    class C implements A, B {
        public void methodA() {
            System.out.println("Method A");
        }
        public void methodB() {
            System.out.println("Method B");
        }
    }
}

```

Here, class C inherits behaviors from both interfaces A and B.

- **Interface Reference**

Interfaces enable runtime polymorphism in Java. An interface reference can refer to objects of different implementing classes, allowing programs to be more dynamic and extensible.

Example:

```

interface Animal {
    void sound();
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}
class Cat implements Animal {
    public void sound() {
        System.out.println("Cat Meows");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();                // Dog Barks
        a = new Cat();
        a.sound();                // Cat Meows
    }
}

```

In this example, the reference variable is of type Animal, but it refers to a Dog object and Cat object at different locations in the program.

- **Interface Inheritance**

One interface can extend another interface using the extends keyword.

Example:

```
interface A {
    void show();
}
interface B extends A {
    void display();
}
class C implements B {
    public void show() {
        System.out.println("Show method");
    }
    public void display() {
        System.out.println("Display method");
    }
}
```

- **Interface with Multiple Inheritance**

An interface can extend multiple interfaces.

```
interface A {
    void show();
}
interface B {
    void display();
}
interface C extends A, B {
    void print();
}
class D implements C {
    public void show() {
        System.out.println("Show method");
    }
    public void display() {
        System.out.println("Display method");
    }
    public void print() {
        System.out.println("Print method");
    }
}
```

Default Methods (Java 8):

- Default methods were introduced in Java 8 to allow interfaces to contain methods with implementation. This feature helps in adding new methods to interfaces without breaking existing classes that already implement them.

- Default method provides **default implementation**, which can be used directly or overridden if needed in subclasses

Syntax:

```
default return_type method_name() {
    // implementation
}
```

Example:

```
interface Test {
    default void show() {
        System.out.println("Default method");
    }
    void display();
}
class Demo implements Test {
    public void display() {
        System.out.println("Display method");
    }
}
```

Here, the class Demo can directly use the default method show() without overriding it.

Static Methods in Interface (Java 8)

- Static methods were introduced in interfaces in Java 8 to **allow utility or helper methods related to the interface to be defined within the interface itself**, improving code organization, cohesion, and supporting functional programming features, without affecting implementing classes.
- Static methods in interfaces belong to the interface itself rather than the implementing classes.
- They cannot be overridden and must be called using the interface name.

Syntax:

```
interface InterfaceName {
    static void methodName() {
        // implementation
    }
}
```

Example:

```
interface Test {
    static void display() {
        System.out.println("Static method");
    }
}
Call using:
```

Test.display();

Note : Static methods cannot be called using objects.

Difference Between Interface and Abstract Class:

Interface	Abstract Class
Uses interface keyword	Uses abstract keyword
Supports multiple inheritance	Does not support multiple inheritance
No constructors	Has constructors
Variables are constants	Variables can be normal
Methods are abstract by default	Can have abstract and concrete methods
Uses implements	Uses extends

Advantages of Interface

- Achieves abstraction
- Supports multiple inheritance
- Improves flexibility
- Supports polymorphism
- Provides loose coupling

When to Use Interface

Use interface when:

- Multiple classes share common behavior
- Multiple inheritance needed
- Full abstraction needed

Example:

Payment → CreditCard, UPI, NetBanking